



DESIGN AND ANALYSIS OF ALGORITHMS

Optimal Binary Search Trees

Reetinder Sidhu

Department of Computer Science and Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

Optimal Binary Search Trees

Reetinder Sidhu

Department of Computer Science and
Engineering

OPTIMAL BINARY SEARCH TREES

UNIT 5: Limitations of Algorithmic Power and Coping with the Limitations

- Dynamic Programming
 - ▶ Computing a Binomial Coefficient
 - ▶ The Knapsack Problem
 - ▶ Memory Functions
 - ▶ Warshall's and Floyd's Algorithms
 - ▶ **Optimal Binary Search Trees**
- Limitations of Algorithmic Power
 - ▶ Lower-Bound Arguments
 - ▶ Decision Trees
 - ▶ P, NP, and NP-Complete, NP-Hard Problems
- Coping with the Limitations
 - ▶ Backtracking
 - ▶ Branch-and-Bound

Concepts covered

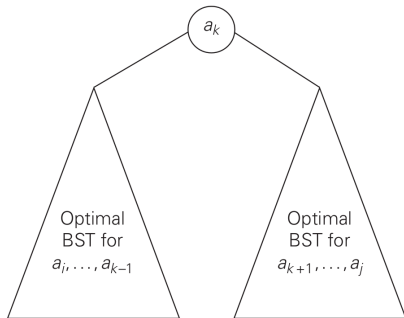
- Optimal Binary Search Trees
 - ▶ Definition
 - ▶ Algorithm
 - ▶ Example

- Given n keys $a_1 < \dots < a_n$ and probabilities $p_1 \dots p_n$ searching for them, find a binary search tree (BST) with a minimum average number of comparisons in successful search
- Since total number of BSTs with n nodes is given by $\frac{C(2n, n)}{n+1}$, which grows exponentially, brute force is hopeless

OPTIMAL BINARY SEARCH TREES

Dynamic Programming

- Let $C[i, j]$ be minimum average number of comparisons made in $T[i, j]$, optimal BST for keys $a_i < \dots < a_j$, where $1 \leq i \leq j \leq n$. Consider optimal BST among all BSTs with some $a_k (i \leq k \leq j)$ as their root; $T[i, j]$ is the best among them



$$C[i, j] = \min_{i \leq k \leq j} (p_k \cdot 1 + \sum_{s=i}^{k-1} p_s \cdot (\text{level of } a_s \text{ in } T_i^{k-1} + 1) + \sum_{s=k+1}^j p_s \cdot (\text{level of } a_s \text{ in } T_{k+1}^j + 1))$$

OPTIMAL BINARY SEARCH TREES

Dynamic Programming

NOT IN SYLLABUS

OPTIMAL BINARY SEARCH TREES

Dynamic Programming

- Simplifying yields the following recurrence:

$$C[i..j] = \min_{i \leq k \leq j} (C(i, k-1) + C(k+1, j)) + \sum_{s=i}^j p_s, \quad 1 \leq i \leq j \leq n$$

NOT IN SYLLABUS

OPTIMAL BINARY SEARCH TREES

Dynamic Programming

- Simplifying yields the following recurrence:

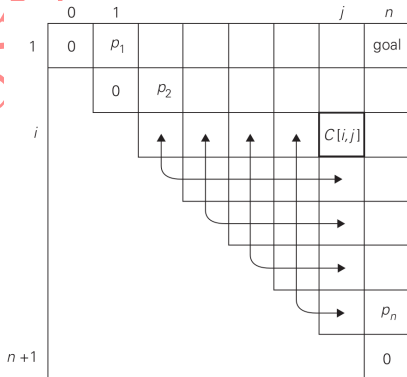
$$C[i,j] = \min_{i \leq k \leq j} (C(i, k-1) + C(k+1, j)) + \sum_{s=i}^j p_s, \quad 1 \leq i \leq j \leq n$$

- Above recurrence implies:

$$C(i, i) = p_i, \quad 1 \leq i \leq n$$

- Above recurrence implies:

$$C(i, i) = p_i, \quad 1 \leq i \leq n$$



OPTIMAL BINARY SEARCH TREES

Example

Key, probability table

key	A	B	C	D
probability	0.1	0.2	0.4	0.3

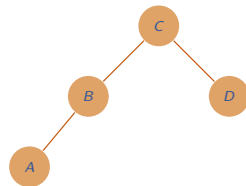
i \ j	0	1	2	3	4
1	0	0.1	0.4	1.1	1.7
2		0	0.2	0.8	1.4
3			0	0.4	1.0
4				0	0.3
5					0

i \ j	0	1	2	3	4
1		1	2	3	3
2			2	3	3
3				3	3
4					4
5					

- Left table filled with recurrence:

$$C[i,j] = \min_{i \leq k \leq j} (C(i, k-1) + C(k+1, j)) + \sum_{s=i}^j p_s, \quad C[i, i] = p_i$$

- Right table, for tree roots, records k values, giving minima



Optimal binary tree

OPTIMAL BINARY SEARCH TREES

Optimal Binary Search Tree Construction

```
1: procedure OPTIMALBST( $P[1 \dots n]$ )
2:   ▷ Finds an optimal binary search tree by dynamic programming
3:   ▷ Input: An array  $P[1 \dots n]$  of search probabilities for a sorted list of  $n$  keys
4:   ▷ Output: Average number of comparisons in successful searches in the
5:   ▷ optimal BST and table  $R$  of subtrees' roots in the optimal BST
6:   for  $i \leftarrow 1$  to  $n$  do
7:      $C[i, i-1] \leftarrow 0$ 
8:      $C[i, i] \leftarrow P[i]$ 
9:      $R[i, i] \leftarrow i$ 
10:   $C[n+1, n] \leftarrow 0$ 
11:  for  $d \leftarrow 1$  to  $n-1$  do
12:    for  $i \leftarrow 1$  to  $n-d$  do
13:       $j \leftarrow i+d$ 
14:       $minval \leftarrow \infty$ 
15:      for  $k \leftarrow i$  to  $j$  do
16:        if  $C[i, k-1] + C[k+1, j] < minval$  then
17:           $minval \leftarrow C[i, k-1] + C[k+1, j]$ 
18:           $kmin \leftarrow k$ 
19:       $R[i, j] \leftarrow kmin$ 
20:       $sum \leftarrow P[i]$ 
21:      for  $s \leftarrow i+1$  to  $j$  do
22:         $sum \leftarrow sum + P[s]$ 
23:       $C[i, j] \leftarrow minval + sum$ 
24:  return  $C[i, n], R$ 
```



OPTIMAL BINARY SEARCH TREES

Optimal Binary Search Tree Construction

```
1: procedure OPTIMALBST( $P[1 \dots n]$ )
2:   ▷ Finds an optimal binary search tree by dynamic programming
3:   ▷ Input: An array  $P[1 \dots n]$  of search probabilities for a sorted list of  $n$  keys
4:   ▷ Output: Average number of comparisons in successful searches in the
5:   ▷ optimal BST and table  $R$  of subtrees' roots in the optimal BST
6:   for  $i \leftarrow 1$  to  $n$  do
7:      $C[i, i-1] \leftarrow 0$ 
8:      $C[i, i] \leftarrow P[i]$ 
9:      $R[i, i] \leftarrow i$ 
10:   $C[n+1, n] \leftarrow 0$ 
11:  for  $d \leftarrow 1$  to  $n-1$  do
12:    for  $i \leftarrow 1$  to  $n-d$  do
13:       $j \leftarrow i+d$ 
14:       $minval \leftarrow \infty$ 
15:      for  $k \leftarrow i$  to  $j$  do
16:        if  $C[i, k-1] + C[k+1, j] < minval$  then
17:           $minval \leftarrow C[i, k-1] + C[k+1, j]$ 
18:           $kmin \leftarrow k$ 
19:       $R[i, j] \leftarrow kmin$ 
20:       $sum \leftarrow P[i]$ 
21:      for  $s \leftarrow i+1$  to  $j$  do
22:         $sum \leftarrow sum + P[s]$ 
23:       $C[i, j] \leftarrow minval + sum$ 
24:  return  $C[i, n], R$ 
```

- Space: $\Theta(n^2)$

OPTIMAL BINARY SEARCH TREES

Optimal Binary Search Tree Construction

```
1: procedure OPTIMALBST( $P[1 \dots n]$ )
2:   ▷ Finds an optimal binary search tree by dynamic programming
3:   ▷ Input: An array  $P[1 \dots n]$  of search probabilities for a sorted list of  $n$  keys
4:   ▷ Output: Average number of comparisons in successful searches in the
5:   ▷ optimal BST and table  $R$  of subtrees' roots in the optimal BST
6:   for  $i \leftarrow 1$  to  $n$  do
7:      $C[i, i-1] \leftarrow 0$ 
8:      $C[i, i] \leftarrow P[i]$ 
9:      $R[i, i] \leftarrow i$ 
10:   $C[n+1, n] \leftarrow 0$ 
11:  for  $d \leftarrow 1$  to  $n-1$  do
12:    for  $i \leftarrow 1$  to  $n-d$  do
13:       $j \leftarrow i+d$ 
14:       $minval \leftarrow \infty$ 
15:      for  $k \leftarrow i$  to  $j$  do
16:        if  $C[i, k-1] + C[k+1, j] < minval$  then
17:           $minval \leftarrow C[i, k-1] + C[k+1, j]$ 
18:           $kmin \leftarrow k$ 
19:       $R[i, j] \leftarrow kmin$ 
20:       $sum \leftarrow P[i]$ 
21:      for  $s \leftarrow i+1$  to  $j$  do
22:         $sum \leftarrow sum + P[s]$ 
23:       $C[i, j] \leftarrow minval + sum$ 
24:  return  $C[i, n], R$ 
```

- Space: $\Theta(n^2)$
- Time: $\Theta(n^3)$, can be reduced to $\Theta(n^2)$ by exploiting monotonicity of entries in root table ($R[i, j]$ is always between $R[i, j-1]$ and $R[i+1, j]$)

OPTIMAL BINARY SEARCH TREES

Optimal Binary Search Tree Construction

```
1: procedure OPTIMALBST( $P[1 \dots n]$ )
2:   ▷ Finds an optimal binary search tree by dynamic programming
3:   ▷ Input: An array  $P[1 \dots n]$  of search probabilities for a sorted list of  $n$  keys
4:   ▷ Output: Average number of comparisons in successful searches in the
5:   ▷ optimal BST and table  $R$  of subtrees' roots in the optimal BST
6:   for  $i \leftarrow 1$  to  $n$  do
7:      $C[i, i-1] \leftarrow 0$ 
8:      $C[i, i] \leftarrow P[i]$ 
9:      $R[i, i] \leftarrow i$ 
10:   $C[n+1, n] \leftarrow 0$ 
11:  for  $d \leftarrow 1$  to  $n-1$  do
12:    for  $i \leftarrow 1$  to  $n-d$  do
13:       $j \leftarrow i+d$ 
14:       $minval \leftarrow \infty$ 
15:      for  $k \leftarrow i$  to  $j$  do
16:        if  $C[i, k-1] + C[k+1, j] < minval$  then
17:           $minval \leftarrow C[i, k-1] + C[k+1, j]$ 
18:           $kmin \leftarrow k$ 
19:       $R[i, j] \leftarrow kmin$ 
20:       $sum \leftarrow P[i]$ 
21:      for  $s \leftarrow i+1$  to  $j$  do
22:         $sum \leftarrow sum + P[s]$ 
23:       $C[i, j] \leftarrow minval + sum$ 
24:  return  $C[i, n], R$ 
```

- Space: $\Theta(n^2)$
- Time: $\Theta(n^3)$, can be reduced to $\Theta(n^2)$ by exploiting monotonicity of entries in root table ($R[i, j]$ is always between $R[i, j-1]$ and $R[i+1, j]$)
- Method can be expanded to include unsuccessful searches